INSTITUTO FEDERAL DE MINAS GERAIS

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

CARLOS HENRIQUE LIMA

Exercício de Aprendizagem – LPI

SÃO JOÃO EVANGELISTA

2025

Exercício 1: Na aula anterior, você implementou uma classe Stack em Java, com operações básicas como empilhar, desempilhar, verificar se está vazia e tratamento de exceções para situações como tentativa de desempilhar de uma pilha vazia. Nesta atividade, adicione um construtor de cópia à classe Stack, de forma que seja possível criar uma nova pilha a partir de outra já existente, copiando todos os elementos na mesma ordem.
Após implementar o construtor, crie uma classe de teste que demonstre seu funcionamento. No método main, crie uma pilha original, adicione alguns elementos e, em seguida, utilize o construtor de cópia para criar uma nova pilha baseada na primeira. Mostre, por meio de saídas no console, que ambas as pilhas contêm os mesmos valores, mas são objetos independentes (ou seja, alterações em uma não afetam a outra).

```
package ExerciciosRevisao.Exercicio1;

public class Main {

    public static void main(String[] args) {

        Stack A = new Stack();

        //try {

            A.push(10);
            A.push(20);
            A.push(30);

            Stack B = new Stack(A);
            A.pop();

            System.out.println("\nAqui é A:");
            A.show();
```

```
            System.out.println("\nAqui é B:");
            B.show();

       /* } catch (Exception e) {
            System.out.println(e.getMessage());
        }*/
    }
}

package ExerciciosRevisao.Exercicio1;

public class Stack {

    private Node top;
    private int size;

    public Stack() { // Criando uma pilha vazia.
        this.top = null;
        this.size = 0;
    }

    public void push(int item) { // Empilhar.
        Node t = new Node(item);
        t.next = this.top;
        this.top = t;
        this.size++;
    }

    public boolean isEmpty() { // Verificar se está vazia.
        return this.top == null;
    }

    public int size() { // Retorna tamanho
        return this.size;
```

```
    }

    public int pop() { // Desempilhar
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }

        int item = this.top.item;
        this.top = this.top.next;
        this.size--;
        return item;
    }

    public int top() { // Retorna o item do topo.
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }
        return this.top.item;
    }

    public Stack(Stack outro) { // Construtor de copia
        Node t = outro.top;
        Stack S = new Stack();

        while (t != null) {
            S.push(t.item);
            t = t.next;
        }
        this.top = null;
        this.size = 0;

        while (!S.isEmpty()) {
```

```
            this.push(S.pop());
        }
    }

    public void show () { // Metodo para imprimir
        Node t = this.top;

        while (t != null) {
           System.out.println(t.item);
            t = t.next;
        }
    }

}

class Node {

    public int item;
    public Node next;

    public Node(int item) {
        this.item = item;
        this.next = null;
    }
}
```

```
Aqui é A:
20
10

Aqui é B:
30
20
10
```

Exercício 2: Amplie a classe Stack criada anteriormente, implementando um construtor que receba um número inteiro n como parâmetro. Esse construtor deve inicializar a pilha vazia e, em seguida, empilhar automaticamente os números inteiros de 1 até n, nessa ordem (ou seja, o número 1 é empilhado primeiro e o número n ficará no topo).

Após implementar o construtor, crie um programa de teste no método main que solicite ao usuário um valor n, crie uma nova pilha utilizando o construtor implementado e exiba os elementos empilhados. Em seguida, desempilhe todos os elementos mostrando a sequência de remoção, para confirmar que a pilha está funcionando corretamente e que os elementos foram empilhados na ordem esperada.

```
package ExerciciosRevisao.Exercicio2;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner cin = new Scanner(System.in);
        int n;

        System.out.println("\nDigite um numero:");
        n = cin.nextInt();
        Stack C = new Stack(n);
        System.out.println("\nElememto(s):");
        C.show();
        System.out.println("\nRemovido(s): ");
        while (!C.isEmpty()) {
            System.out.println(C.pop());
        }

        cin.close();
    }
}


package ExerciciosRevisao.Exercicio2;

public class Stack {

    private Node top;
    private int size;
```

```
    public Stack() { // Criando uma pilha vazia.
        this.top = null;
        this.size = 0;
    }

    public void push(int item) { // Empilhar.
        Node t = new Node(item);
        t.next = this.top;
        this.top = t;
        this.size++;
    }

    public boolean isEmpty() { // Verificar se está vazia.
        return this.top == null;
    }

    public int size() { // Retorna tamanho
        return this.size;
    }

    public int pop() { // Desempilhar
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }

        int item = this.top.item;
        this.top = this.top.next;
        this.size--;
        return item;
    }

    public int top() { // Retorna o item do topo.
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }
        return this.top.item;
    }

    public Stack(Stack outro) { // Construtor de copia
        Node t = outro.top;
        Stack S = new Stack();

        while (t != null) {
            S.push(t.item);
            t = t.next;
        }
        this.top = null;
        this.size = 0;
```

```
        while (!S.isEmpty()) {
            this.push(S.pop());
        }
    }

    public void show() { // Metodo para imprimir
        Node t = this.top;

        while (t != null) {
            System.out.println(t.item);
            t = t.next;
        }
    }

    public Stack(int n) { // Criando pilha aprtir de um n
        this.top = null;
        this.size = 0;

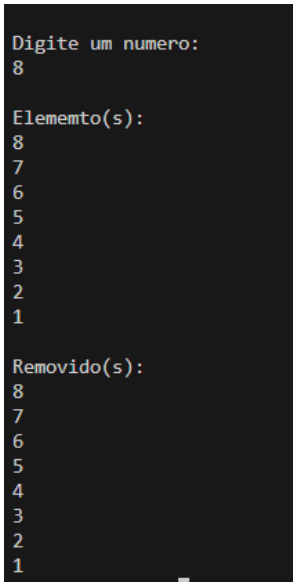
        for(int i = 1; i <= n; i++)
        {
            this.push(i);
        }
    }

}

class Node {

    public int item;
    public Node next;

    public Node(int item) {
        this.item = item;
        this.next = null;
    }
}
```



Exercício 3: Agora, modifique a a classe Stack já desenvolvida, incluindo um método estático chamado equals que receba como parâmetros dois objetos do tipo Stack e retorne um valor lógico (true ou false) indicando se as pilhas são iguais ou diferentes. Duas pilhas devem ser consideradas iguais se possuírem o mesmo número de elementos e se todos os elementos corresponderem exatamente na mesma ordem, do topo até a base. O método não deve alterar o conteúdo das pilhas comparadas; caso seja necessário percorrer os elementos, utilize pilhas auxiliares para preservar o estado original.

No método main, crie duas instâncias da classe Stack, empilhando alguns valores em cada uma delas (você pode utilizar o construtor que empilha automaticamente os números de 1 até n). Em seguida, invoque o método equals para comparar as duas pilhas e exiba uma mensagem informando se elas são iguais ou diferentes.

```
package ExerciciosRevisao.Exercicio3;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner cin = new Scanner(System.in);
```

```
        int n, m;

        System.out.println("\nDigite um numero:");
        n = cin.nextInt();
        Stack D = new Stack(n);

        System.out.println("\nDigite um numero:");
        m = cin.nextInt();
        Stack E = new Stack(m);

        System.out.println("\nElememto(s) D:");
        D.show();
        System.out.println("\nElememto(s) E:");
        E.show();

        if (Stack.equals(D, E)) {
            System.out.println("\nAs Pilhas são
verdadeiras");
        } else {
            System.out.println("\nAs pilhas são
diferentes.");
        }

        cin.close();
    }
}


package ExerciciosRevisao.Exercicio3;

public class Stack {

    private Node top;
    private int size;

    public Stack() { // Criando uma pilha vazia.
        this.top = null;
        this.size = 0;
    }

    public void push(int item) { // Empilhar.
        Node t = new Node(item);
        t.next = this.top;
        this.top = t;
        this.size++;
    }

    public boolean isEmpty() { // Verificar se está vazia.
        return this.top == null;
    }
```

```
    public int size() { // Retorna tamanho
        return this.size;
    }

    public int pop() { // Desempilhar
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }

        int item = this.top.item;
        this.top = this.top.next;
        this.size--;
        return item;
    }

    public int top() { // Retorna o item do topo.
        if (this.isEmpty()) {
            throw new IllegalStateException("A pilha está
vazia.");
        }
        return this.top.item;
    }

    public Stack(Stack outro) { // Construtor de copia
        Node t = outro.top;
        Stack S = new Stack();

        while (t != null) {
            S.push(t.item);
            t = t.next;
        }
        this.top = null;
        this.size = 0;

        while (!S.isEmpty()) {
            this.push(S.pop());
        }
    }

    public void show() { // Metodo para imprimir
        Node t = this.top;

        while (t != null) {
            System.out.println(t.item);
            t = t.next;
        }
    }

    public Stack(int n) { // Criando pilha apartir de um n
        this.top = null;
```

```
        this.size = 0;

        for (int i = 1; i <= n; i++) {
            this.push(i);
        }
    }

    public static boolean equals(Stack A, Stack B) { //
Verificando se duas pilhas são iguais

        if (A.size != B.size) {
            return false;
        } else {
            Node t1 = A.top;
            Node t2 = B.top;

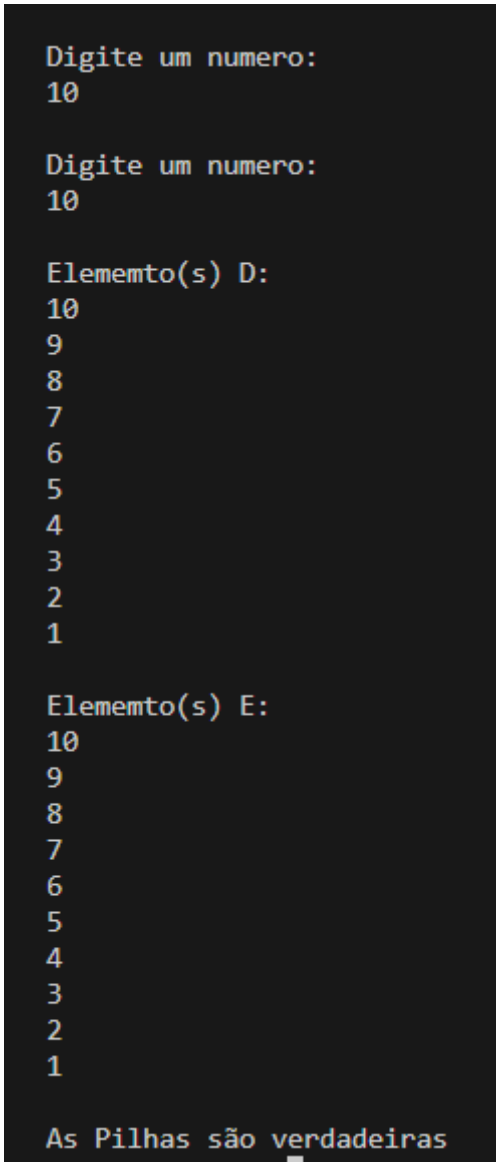
            while (t1 != null) {
                if (t1.item != t2.item) {
                    return false;
                }
                t1 = t1.next;
                t2 = t2.next;
            }
        }
        return true;
    }

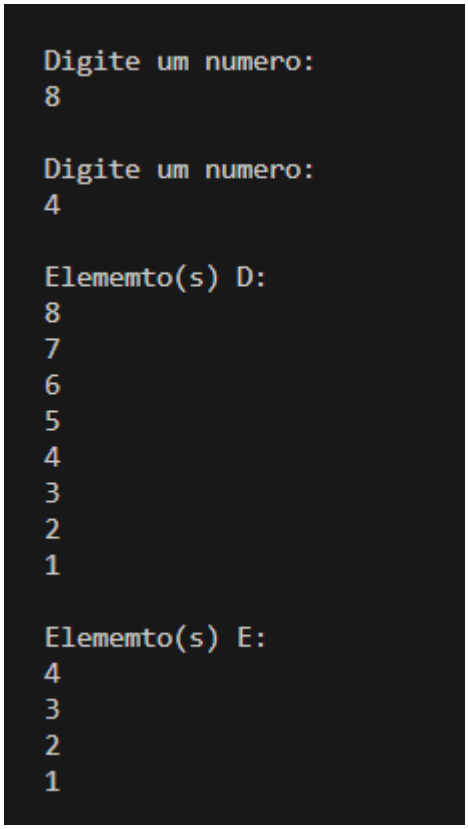
}

class Node {

    public int item;
    public Node next;

    public Node(int item) {
        this.item = item;
        this.next = null;
    }
}
```





Exercício 4: Crie um programa em Java com classes de um sistema simples de gerenciamento de alunos de uma universidade. Cada aluno deve ser representado por uma classe contendo atributos como nome, matricula, curso e idade. Utilize o conceito de encapsulamento para garantir que os atributos só possam ser acessados por meio de métodos públicos (getters e setters). Em seguida, crie um método para exibir as informações do aluno de forma formatada. No método main, instancie pelo menos três objetos da classe Aluno e exiba seus dados na tela.

```
package ExerciciosRevisao.Exercicio4;
```

```
public class Main {

    public static void main(String[] args) {

        Aluno A = new Aluno();
        Aluno B = new Aluno();
        Aluno C = new Aluno();

        A.setNome("Carlos");
        A.setIdade(20);
        A.setMatricula(1);
        A.setCurso("Sistema de Informações");

        B.setNome("Suzane");
        B.setIdade(19);
        B.setMatricula(2);
        B.setCurso("Administração");

        C.setNome("Felipe");
        C.setIdade(20);
        C.setMatricula(3);
        C.setCurso("Engenharia Florestal");

        A.informacoes();
        B.informacoes();
        C.informacoes();
    }
}


package ExerciciosRevisao.Exercicio4;

public class Aluno {

    private String nome;
    private int matricula;
    private String curso;
    private int idade;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public int getMatricula() {
        return matricula;
    }
```

```
    public void setMatricula(int matricula) {
        this.matricula = matricula;
    }

    public String getCurso() {
        return curso;
    }

    public void setCurso(String curso) {
        this.curso = curso;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    public void informacoes() {
        System.out.println("Aluno: " + getNome());
        System.out.println("Idade: " + getIdade());
        System.out.println("Matricula: " + getMatricula());
        System.out.println("Curso: " + getCurso());
    }

}
```

```
Aluno: Carlos
Idade: 20
Matricula: 1
Curso: Sistema de Informações
Aluno: Suzane
Idade: 19
Matricula: 2
Curso: Administração
Aluno: Felipe
Idade: 20
Matricula: 3
Curso: Engenharia Florestal
```

Exercício 5: Amplie o exercício anterior incluindo construtores sobrecarregados na classe Aluno: um construtor sem parâmetros, um que receba apenas nome e matrícula, e outro que receba todos os atributos. No programa principal, crie objetos utilizando cada um dos construtores e verifique como a

sobrecarga facilita a criação de instâncias com diferentes quantidades de informações.

```
package ExerciciosRevisao.Exercicio5;

public class Main {
    public static void main(String[] args) {

        Aluno a1 = new Aluno();
        a1.setNome("Carlos");
        a1.setIdade(20);
        a1.setMatricula(1);
        a1.setCurso("Sistemas de Informação");

        Aluno a2 = new Aluno("Suzane", 2);
        a2.setCurso("Administração");
        a2.setIdade(19);

        Aluno a3 = new Aluno("Felipe", 3, "Engenharia
Florestal", 20);

        a1.informacoes();
        a2.informacoes();
        a3.informacoes();
    }
}


package ExerciciosRevisao.Exercicio5;

public class Aluno {

    private String nome;
    private int matricula;
    private String curso;
    private int idade;

    public Aluno() {
        System.out.println("Construtor sem parâmetros
chamado!\n");
    }

    public Aluno(String nome, int matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }

    public Aluno(String nome, int matricula, String curso,
int idade) {
```

```
        this.nome = nome;
        this.matricula = matricula;
        this.curso = curso;
        this.idade = idade;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public int getMatricula() {
        return matricula;
    }

    public void setMatricula(int matricula) {
        this.matricula = matricula;
    }

    public String getCurso() {
        return curso;
    }

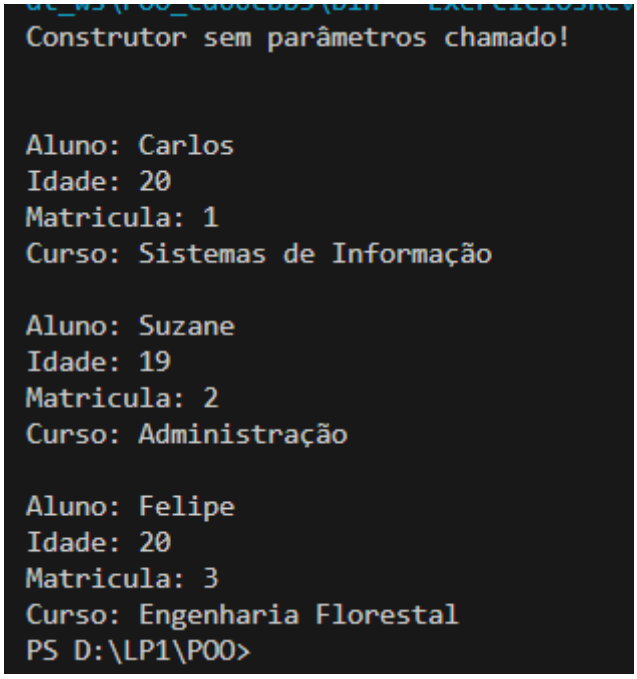
    public void setCurso(String curso) {
        this.curso = curso;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    public void informacoes() {
        System.out.println("\nAluno: " + getNome());
        System.out.println("Idade: " + getIdade());
        System.out.println("Matricula: " + getMatricula());
        System.out.println("Curso: " + getCurso());
    }

}
```



Exercício 6: Crie uma classe chamada Universidade com um atributo estático que armazene o nome da instituição. Faça com que cada objeto da classe Aluno utilize esse atributo para exibir, junto de suas informações, o nome da universidade. Além disso, crie um método estático na classe Universidade que permita alterar o nome da instituição e verifique como essa alteração afeta todos os objetos já criados.

```
package ExerciciosRevisao.Exercicio6;

public class Main {

    public static void main(String[] args) {

        Aluno a1 = new Aluno("Carlos", 1, "Sistemas de
Informação", 20);
        Aluno a2 = new Aluno("Suzane", 2, "Administração",
19);

        System.out.println("\nAntes da mudança:\n");
        a1.informacoes();
        a2.informacoes();

        Universidade.setNomeInstituicao("Universidade Federal
de Minas Gerais - UFMG");

        System.out.println("\nDepois da mudança:\n");
```

```
        a1.informacoes();
        a2.informacoes();
    }
}


package ExerciciosRevisao.Exercicio6;

public class Aluno {

    private String nome;
    private int matricula;
    private String curso;
    private int idade;

    public Aluno(String nome, int matricula, String curso,
int idade) {
        this.nome = nome;
        this.matricula = matricula;
        this.curso = curso;
        this.idade = idade;
    }

    public void informacoes() {
        System.out.println("Aluno: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Matrícula: " + matricula);
        System.out.println("Curso: " + curso);
        System.out.println("Universidade: " +
Universidade.getNomeInstituicao());
        System.out.println();
    }
}


package ExerciciosRevisao.Exercicio6;

public class Universidade {

    private static String nomeInstituicao = "IFMG - Campus
São João Evangelista";

    public static String getNomeInstituicao() {
        return nomeInstituicao;
    }

    public static void setNomeInstituicao(String novoNome) {
        nomeInstituicao = novoNome;
    }
}
```

```
Antes da mudança:

Aluno: Carlos
Idade: 20
Matrícula: 1
Curso: Sistemas de Informação
Universidade: IFMG - Campus São João Evangelista

Aluno: Suzane
Idade: 19
Matrícula: 2
Curso: Administração
Universidade: IFMG - Campus São João Evangelista


Depois da mudança:

Aluno: Carlos
Idade: 20
Matrícula: 1
Curso: Sistemas de Informação
Universidade: Universidade Federal de Minas Gerais - UFMG

Aluno: Suzane
Idade: 19
Matrícula: 2
Curso: Administração
Universidade: Universidade Federal de Minas Gerais - UFMG
```

Exercício 7: Explique, com suas próprias palavras, os conceitos de checked exceptions e unchecked exceptions na linguagem Java, destacando as diferenças entre esses dois tipos de exceção e apresentando exemplos que ilustrem cada caso. Em seguida, discuta de que forma a distinção entre exceções verificadas e não verificadas pode contribuir para a adoção de boas práticas de programação, como a escrita de códigos mais seguros, organizados e fáceis de manter. Seu texto deve evidenciar a importância do tratamento adequado de erros para garantir maior confiabilidade e estabilidade na execução de aplicações Java.

R: Em Java, as exceções são erros que ocorrem durante a execução do programa e se dividem em checked e unchecked exceptions. As checked exceptions devem ser tratadas obrigatoriamente com try/catch ou declaradas com throws, pois representam erros previstos,

como falhas ao acessar arquivos ou conexões. Já as unchecked exceptions ocorrem em tempo de execução e não precisam ser tratadas, sendo causadas por erros de lógica, como NullPointerException ou ArrayIndexOutOfBoundsException. Essa distinção incentiva boas práticas de programação, tornando o código mais seguro, organizado e confiável. O tratamento adequado de exceções é essencial para garantir maior estabilidade e evitar falhas inesperadas nas aplicações Java.

```
Exemplos:

try {
    FileReader leitor = new FileReader("arquivo.txt");
} catch (FileNotFoundException e) {
    System.out.println("Arquivo não encontrado!");
}


int[] numeros = {1, 2, 3};
System.out.println(numeros[5]); // Lança
ArrayIndexOutOfBoundsException
```

Exercício 8: Explique, com suas próprias palavras, a diferença entre o uso das palavras-chave throw e throws no contexto do tratamento de exceções em Java.

R: Em Java, throw é usado dentro de um método para lançar uma exceção quando ocorre um erro, enquanto throws é usado na declaração do método para indicar que ele pode lançar exceções e que o tratamento deve ser feito por quem o chamar.